

How I Seeded Hindsight With Cross-Platform Support Tickets

A support agent can turn yesterday's untested suggestion into today's confident answer if both enter the same memory. I built Fleet Command around a stricter rule: a resolution has to earn its place in Hindsight before the assistant can reuse it.

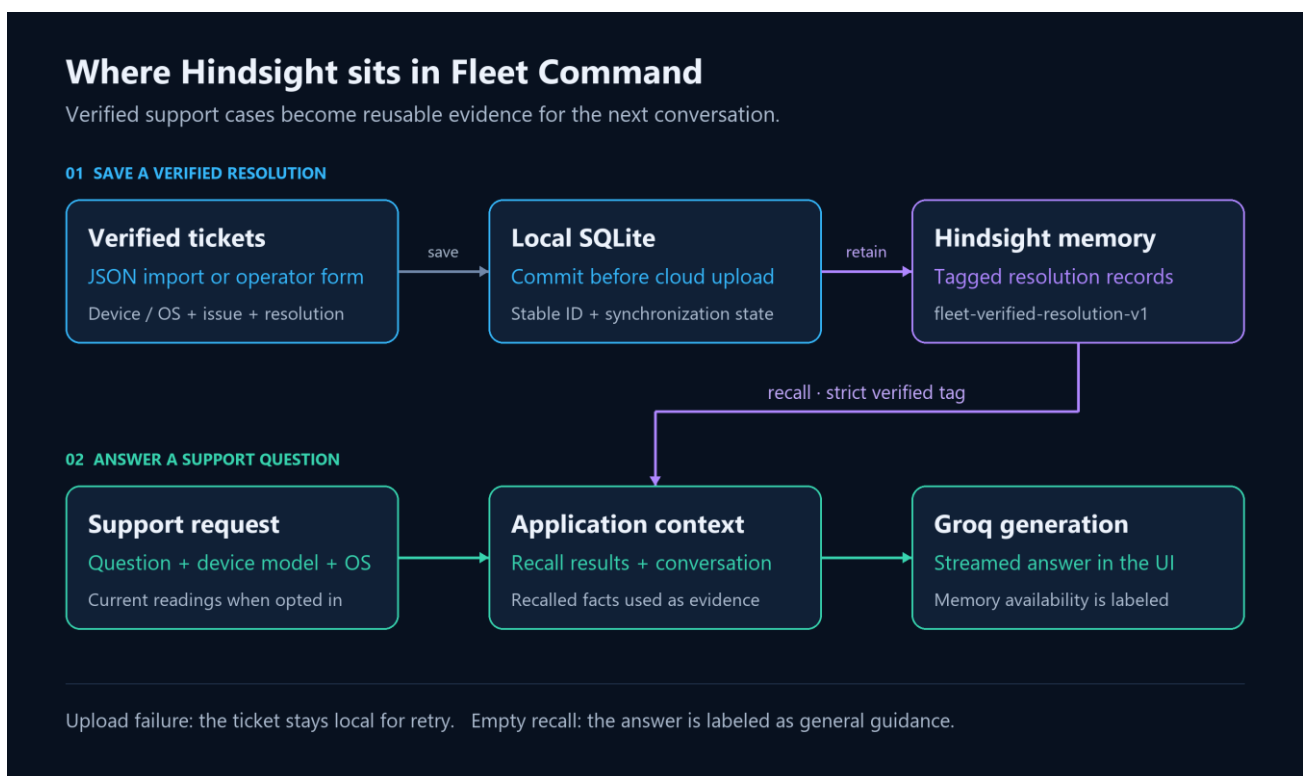
The interesting part of seeding memory turned out to be the boundary around the seed data. I needed to preserve the conditions under which a fix worked, survive interrupted uploads, and keep a retrieved case distinguishable from general troubleshooting advice.

That led to a deliberately explicit workflow: validate a ticket, commit it locally, retain it in Hindsight, and recall only records admitted through the verified-resolution path.

A support case is the unit of memory

Fleet Command combines a Streamlit support interface, local hardware diagnostics, a Groq-hosted language model, and Hindsight memory. An operator can inspect a machine, ask for troubleshooting help, and save a resolution after verifying it on the affected device.

The code separates those responsibilities. `app.py` owns the conversation and resolution-entry workflow. `telemetry.py` collects operating-system readings. `services.py` handles persistence and remote calls. `seed_data.py` imports existing tickets through the same persistence functions used by the interface.



The write path preserves verified cases; the read path supplies relevant evidence to the conversation.

Hindsight provides the memory operations that connect previous support work to a new question. The [open-source Hindsight agent memory repository](#) describes retain, recall, and reflect; my application uses retain and recall directly. Groq generates the response after the application assembles the current hardware context and any recalled resolutions.

I keep three kinds of information separate: what the machine reports now, what someone said in a conversation, and what an operator verified in a previous case. Each has a different lifetime and a different claim to authority.

A CPU reading is an observation at a particular time. A chat reply may contain a proposed next step. A verified ticket records a resolution under stated conditions. Only the last category enters the reusable resolution workflow.

Cross-platform tickets need enough context to stay specific

The import format requires a device target, an issue, a resolution, and an explicit verification flag. The device target can include the model and operating system; the issue and resolution retain the conditions that explain when the fix applies.

Consider two tickets in an import file. One describes a Windows laptop whose VPN disconnects when switching from home Wi-Fi to a mobile hotspot. Another describes a MacBook that cannot resolve internal service names on the office network.

Both might later be described as “the network is broken.” That phrase alone throws away almost everything useful about the previous work. The network transition matters in the first case. The office network and name-resolution behavior matter in the second.

I want those details to survive ingestion. A stored instruction such as “reset networking” is too broad to be useful. A record that preserves the affected platform, triggering condition, observed failure, and verified procedure gives the assistant something it can evaluate against the next request.

This is how I approached cross-platform memory: keep the record format consistent while retaining the platform-specific facts inside it. Uniform storage should not erase differences in applicability.

The distinction between a temporary conversation and knowledge carried across interactions is central to [Vectorize’s explanation of agent memory](#). In Fleet Command, the durable knowledge is the resolution record. The chat is one place where that knowledge gets used.

I validate the entire import before writing anything

The importer accepts a JSON array and checks every ticket before entering its write loop. These are the admission checks in `seed_data.py`:

```
for index, ticket in enumerate(tickets, 1):
    if not isinstance(ticket, dict) or ticket.get("verified") is not True:
        raise ValueError(f"Ticket {index}: verified must be true.")
    for field in ("device_target", "issue", "resolution"):
        if not isinstance(ticket.get(field), str) or not ticket[field].strip():
            raise ValueError(f"Ticket {index}: {field} is required.")
```

The strict Boolean check is intentional. A string containing “false” is truthy in Python. Accepting any truthy value would make a formatting mistake look like verification.

Validating the whole file also avoids an irritating operational failure: importing the first several records, discovering a malformed ticket halfway through, and leaving the operator to work out which records were accepted.

This does not make the remote upload transactional. Network failures can still produce a partially synchronized batch. It separates malformed-input failures, which I can reject before writing, from transport failures, which need recovery after writing.

The verification flag is an assertion by the person preparing the ticket. Software can enforce that the assertion exists; it cannot establish that the procedure actually fixed the machine. The interface reinforces that distinction by requiring the operator to confirm verification when saving a resolution.

I deliberately avoid retaining every assistant response. Otherwise, generated advice could return through retrieval carrying the apparent authority of accumulated support knowledge.

Local persistence gives an upload failure somewhere to land

A verified fix should survive a cloud timeout. I therefore save the record to SQLite before attempting Hindsight retention.

The local save assigns an identity derived from the normalized fields:

```
fields = [device.strip(), issue.strip(), resolution.strip()]
if not all(fields):
    raise ValueError("Device, issue, and verified resolution are required.")
doc_id = hashlib.sha256(json.dumps(fields).encode()).hexdigest()
with connection(db_path) as db:
    db.execute("INSERT OR IGNORE INTO resolutions VALUES (?, ?, ?, ?, ?, 0)",
               (doc_id, *fields, datetime.now(timezone.utc).isoformat()))
```

Saving the same field values twice produces the same local identity. The primary key and `INSERT OR IGNORE` prevent a second local row for that content. The initial synchronization state is zero.

This is exact-content deduplication. Two differently worded descriptions of the same incident can still produce different IDs, and changing a resolution produces a new ID. I do not ask a hash to decide whether two procedures mean the same thing.

Once the local commit succeeds, synchronization can fail without losing the ticket. The interface reports that the record is saved locally with cloud synchronization pending, and exposes a retry action. The importer reports the successful count and returns a nonzero exit status when some uploads fail.

That distinction matters during recovery. "Saved" and "available for recall" are different states. Combining them into one success message makes operators investigate the wrong part of the system.

Retention and recall share an explicit admission marker

When synchronization runs, it retrieves the local record and calls Hindsight with the stable document ID and a dedicated tag:

```
with memory_connection() as client:
    client.retain(
        bank_id=BANK_ID,
        content=(
            f"Device: {row['device']} | Issue: {row['issue']} | "
            f"Resolution: {row['resolution']}"
        ),
        document_id=doc_id,
        tags=[VERIFIED_TAG],
    )
db.execute("UPDATE resolutions SET synced=1 WHERE id=?", (doc_id,))
```

The synchronization flag changes after retention returns successfully. If the call fails, the local record remains pending. A retry uses the same document identity, and a locally synchronized record is skipped on later attempts.

There is still a distributed-systems boundary here. Hindsight can accept a request before the client receives its acknowledgment. I would not call the combination of a remote API and a SQLite transaction an exactly-once operation. Stable identity makes retries deliberate; it does not make the two systems share a commit.

The [Hindsight documentation for retaining and recalling agent memory](#) covers the operations behind this integration. The application's contribution is deciding which support records to retain and how to recover when a call does not finish cleanly.

Recall uses the same admission marker:

```
with memory_connection() as client:
    result = client.recall(
        bank_id=BANK_ID, query=query,
        tags=[VERIFIED_TAG], tags_match="all_strict", max_tokens=2048,
    )
    return "\n\n".join(item.text for item in result.results if item.text)
```

The tag is `fleet-verified-resolution-v1`. Using strict matching excludes untagged records from this retrieval path. That lets the bank contain other material without silently making all of it eligible as verified support guidance.

A tag is an application-level selection rule, not proof of correctness or an authorization system. Its value comes from applying the same rule at ingestion and retrieval.

The return value also matters. I extract the text of the returned facts rather than stringify the entire response object. An empty result produces an empty string, allowing the caller to distinguish “no recalled fix” from “some serialized object exists.”

A recalled case still has to fit the current problem

The recall query includes the current device model, operating system, and user question. That gives Hindsight context for finding applicable support history.

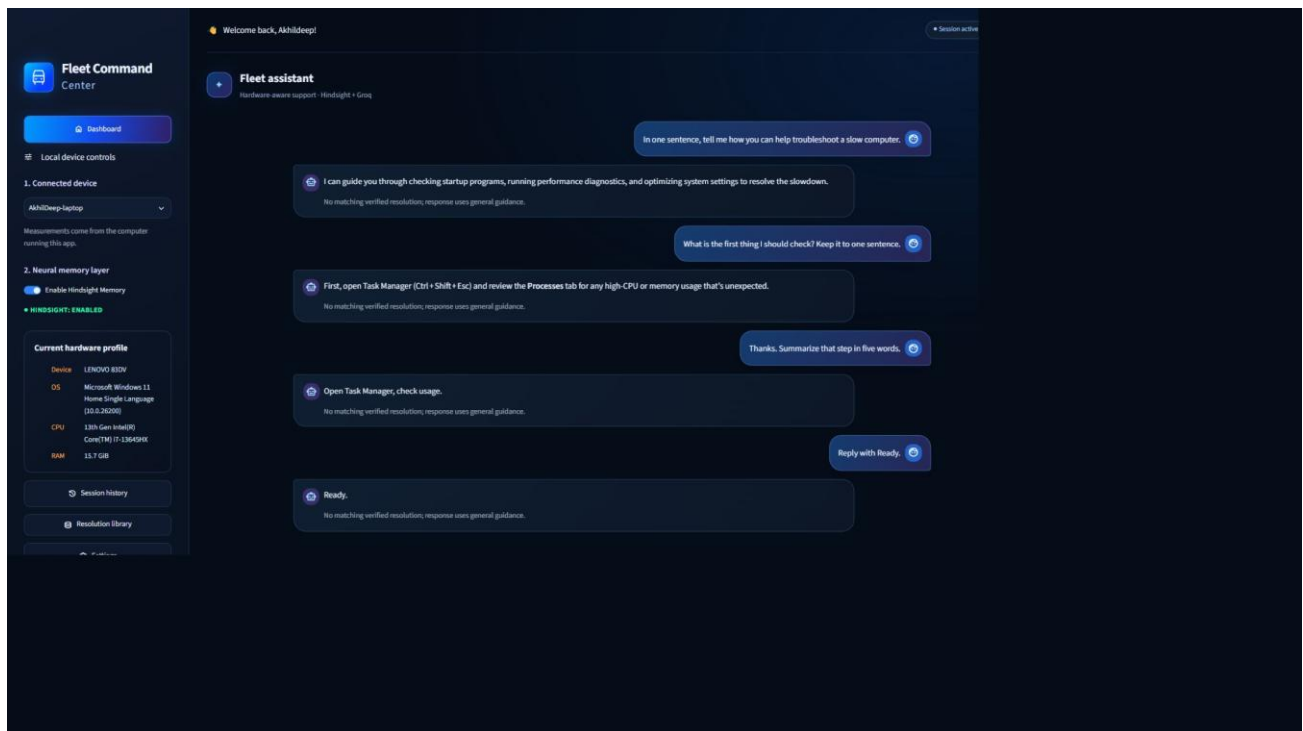
Including those values in a query does not enforce an OS compatibility filter. I treat the retrieved records as candidate evidence. The stored conditions still need to match the problem described by the user.

For the Windows VPN example, a useful interaction starts with something specific: “The VPN disconnects whenever I switch to my hotspot.” The application combines that symptom with the hardware profile before recall. If the relevant verified case is returned, the assistant has the documented procedure available when composing its answer.

For the MacBook example, “internal sites do not resolve on office Wi-Fi” should lead the conversation toward the conditions recorded in that case. Similar networking vocabulary is insufficient reason to reuse a Windows adapter procedure.

These examples describe how the ticket format supports an interaction. They are not a measured claim about retrieval accuracy.

I also make absence visible. When recall returns no text, the response carries a notice explaining that it uses general guidance. When recall fails, the conversation identifies that failure separately. Finding no applicable resolution and failing to query memory deserve different explanations.



The conversation keeps follow-up context; each reply labels the absence of a matching verified resolution.

Hindsight’s practical benefit here is that I can add verified operational knowledge independently of the model. Updating the resolution library changes what the next support request can retrieve without changing the generation code.

I tested recovery behavior before claiming support outcomes

The results I can defend are behavioral. The regression suite checks that a failed synchronization preserves the local ticket, a later retry marks it synchronized, and another retry does not issue an additional write once that local state is recorded. It also checks that recall requests strict verified-tag matching and that an unverified import fails before any save.

The cloud writes in those tests are mocked, so they establish application control flow rather than service availability. Separate live checks exercised Hindsight access and Groq inference.

The conversation tests cover another useful boundary: losing memory access must not erase the user's request. The assistant can continue with clearly labeled general guidance. Stream interruption tests preserve partial replies and allow the user to continue afterward.

I have no ticket-deflection percentage or time-saved benchmark to attach to these checks. Their purpose is narrower and concrete: keep verified work recoverable and make the basis of an answer visible.

Four decisions I would repeat

1. **Preserve the circumstances of a resolution.** Platform, triggering conditions, and the verified procedure determine whether a case is reusable. They deserve to travel together through ingestion and recall.
2. **Make memory admission explicit.** A required verification assertion and a shared ingestion/retrieval tag give the application a clear rule for what it treats as support knowledge.
3. **Separate local durability from remote availability.** Commit the record before uploading it, expose pending synchronization, and retry with a stable identity. Operators need to understand which step succeeded.
4. **Expose the basis of the answer.** A recalled resolution, an empty search, and an unavailable memory service are different situations. The interface should preserve those distinctions even when the model can produce fluent text in all three.

The design decision I value most is the write boundary. Hindsight makes verified cases available across conversations, but the application decides which cases deserve that role. Every useful future answer starts with preserving the conditions and evidence behind a fix someone already established.